

# O.S.M.U Keyword Researcher

## 사용자 매뉴얼

v2 — 로컬 엑셀(.xlsx) + 구글 시트 양방향 동기화 추가

발행일: 2026년 05월 02일

버전: v0.2.0

대상: 비개발자 운영자 / 개발자 / PM

# Claude가 한 작업 ↔ 사용자가 직접 해야 할 작업

이 매뉴얼은 ‘이미 끝난 작업’과 ‘본인이 직접 해야 할 작업’을 구분해서 보여줍니다. 체크박스는 Claude가 끝낸 것, 은 본인이 손수 해야 하는 단계입니다.

## 프로젝트 구축 — Claude가 한 일

- [Claude 완료] 본체 모듈(키워드 평가/폴/추천/연금술/prune) 9개 파일 작성
- [Claude 완료] GUI(Streamlit) 5개 화면 + 홈 대시보드 작성
- [Claude 완료] 양방향 동기화 백엔드(MirrorStorage) + 로컬 엑셀(.xlsx) 백엔드 추가
- [Claude 완료] 단위 테스트 11종 + 시연 시나리오 ① · ② 작성 — 모두 통과
- [Claude 완료] GitHub Actions 자동 테스트 워크플로(.github/workflows/tests.yml) 작성
- [Claude 완료] Codespace prebuild 친화 devcontainer + on-create.sh / post-start.sh 작성
- [Claude 완료] .env 자동 복사 + Codespace Secret 자동 펼침 스크립트 작성
- [Claude 완료] .gitignore 작성 (자격증명 · 데이터 자동 제외)
- [Claude 완료] Mac/Windows 더블클릭 실행 스크립트(run\_mac.command / run\_windows.bat) 작성
- [Claude 완료] 이 매뉴얼 PDF + README 작성

## 최종 사용자가 직접 해야 할 일

- [사용자 작업] Mac/Windows에 Python 3.10 이상 설치 (한 번만)
- [사용자 작업] 프로젝트 폴더에서 가상환경 만들고 의존성 설치
- [사용자 작업] GitHub 계정 만들고 빈 저장소 생성 → 첫 푸시
- [사용자 작업] (선택) Google Cloud Console에서 서비스 계정 자격증명 발급
- [사용자 작업] (선택) 빈 구글 시트 만들고 서비스 계정 이메일에 ‘편집자’ 권한 공유
- [사용자 작업] (선택) Codespace Prebuild 활성화 (Settings → Codespaces)
- [사용자 작업] (선택) Codespace Secret 등록 (구글 시트 자동 연결용)

### 최소 동작 확인

‘사용자 작업’의 첫 두 줄(Python 설치 + 의존성 설치)만 하면 GUI가 동작합니다. 로컬 엑셀(.xlsx)만 사용하는 경우 구글 시트 관련 설정은 모두 건너뛸 수 있어요.

# 목차

1. 투 트랙 관리란? (로컬 엑셀 + 구글 시트)
2. 설치하기 — Mac
3. 설치하기 — Windows
4. GUI 사용 가이드 — 5개 화면
5. 로컬 엑셀로만 관리하기 (가장 간단)
6. 구글 시트와 양방향 동기화 — 단계별 가이드
7. GitHub에 올리기
8. GitHub Codespaces로 실행하기
9. GitHub Actions 자동 테스트
10. Codespace Prebuild & Secret 관리
11. 자주 만나는 문제 (FAQ)
12. 확장 방향 — 다음 단계

# 1. 투 트랙 관리란?

수익형 블로그 운영자는 ‘ 혼자서 빠르게 작업 ’ 도 해야 하고, ‘ 팀과 공유 ’ 도 해야 합니다. 이 두 요구를 모두 만족시키기 위해 본 시스템은 두 갈래의 저장 방식을 동시에 사용할 수 있습니다.

| 트랙              | 역할             | 특징                                                 |
|-----------------|----------------|----------------------------------------------------|
| 로컬<br>엑셀(.xlsx) | 내 컴퓨터에서<br>빠르게 | Excel/Numbers에서 더블클릭으로 열림. 인터넷<br>없이 동작. 가장 빠름.    |
| ☁ 구글 시트         | 팀 공유 · 협업      | 동료가 직접 시트에서 수정 → 자동 동기화.<br>어디서나 접근.               |
| 양방향 동기화         | 두 트랙 합치기       | ‘ 설정 ’ 에서 mirror 모드 선택. 모든 변경이<br>로컬+시트 양쪽에 자동 반영. |

## 어떤 모드를 골라야 하나요?

[사용자 작업] 혼자 사용 + 인터넷 신경 안 씀 → ‘ 로컬 엑셀(.xlsx) ’ 만 사용 (5장  
참고)

[사용자 작업] 팀과 공유 + 어디서나 접근 → ‘ 구글 시트 ’ + 양방향 동기화 (6장 참고)

[사용자 작업] 둘 다 원함 → 양방향 동기화 모드 (권장 — 6장 참고)

## 2. 설치하기 — Mac

### 준비물

Python 3.10 이상이 설치돼 있어야 합니다. 터미널에서 `python3 --version` 실행 시 3.10 이상이면 OK. 없으면 `python.org` 또는 `brew install python@3.11`.

### 2-1. 터미널 열고 프로젝트 폴더로 이동

```
cd ~/Desktop/osmu_keyword_researcher
```

Finder에서 폴더를 터미널 창 위로 드래그하면 경로가 자동 입력됩니다.

### 2-2. 가상환경 + 의존성 설치 (한 번만)

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pip install -r ui/requirements.txt
pip install -e .
```

### 2-3. GUI 실행

```
python main.py
```

### 2-4. 다음 실행부터

```
cd ~/Desktop/osmu_keyword_researcher
source .venv/bin/activate && python main.py
```

### 3. 설치하기 — Windows

#### 준비물

python.org 에서 Python 3.10+ 설치. 설치 마법사에서 ‘ Add Python to PATH ’ 체크박스 반드시 켜기.

```
cd C:\Users\<■■>\Desktop\osmu_keyword_researcher
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
pip install -r ui\requirements.txt
pip install -e .
python main.py
```

PowerShell 실행정책 차단 시: Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

## 4. GUI 사용 가이드 — 5개 화면

좌측 사이드바 메뉴에서 화면을 전환합니다. 사용 흐름은 ① 키워드 생성 → ③ 추천 · 선택 두 단계가 핵심입니다.

### ① 키워드 생성

주제 한 단어 입력 → ‘키워드 만들기’. 점수에 따라 **황금** / **좋은** / **보통** 라벨이 붙습니다. ‘이 키워드 강화하기’로 보통 점수 키워드를 long-tail 변형으로 자동 강화할 수 있어요.

### ② 키워드 풀 관리

저장된 모든 키워드를 한 곳에서. 필터/정렬/체크박스 일괄작업, 그리고 다음 두 신규 기능이 추가됐습니다:

[Claude 완료] 엑셀로 내려받기 — 현재 풀을 .xlsx 로 즉시 다운로드

[Claude 완료] ☁ 시트에서 새로그침 — 시트에서 직접 수정한 변경 가져오기

[Claude 완료] 시트로 업로드 — 로컬 변경을 시트로 일괄 푸시

### ③ 추천 & 선택

재사용 간격(쿨다운)이 적용된 키워드만 카드 형태로 보여줍니다. ‘이 키워드로 글 쓰기’ 한 번 누르면 그 키워드는 풀에서 빠지고 콘텐츠 작성 대기열에 등록됩니다.

### ④ 설정

| 설정 항목        | 기본값  | 의미                             |
|--------------|------|--------------------------------|
| 저장 위치        | auto | auto / mirror / sheets / local |
| 로컬 파일 형식     | xlsx | Excel(.xlsx) 또는 CSV            |
| 최대 보관 개수     | 50   | 풀 최대 크기                        |
| 재평가 주기(일)    | 14   | 이 기간 지난 키워드 자동 재평가/만료          |
| 동일 주제 재사용 간격 | 7    | 쿨다운 — 같은 주제 너무 자주 X            |

### ⑤ 로그 / 상태

현재 백엔드 / 평가 방식 / 자격증명 유무 / 동기화 상태 / 최근 작업 로그를 확인합니다.

## 5. 로컬 엑셀로만 관리하기 (가장 간단)

Google 계정/자격증명/시트 ID 등 어떤 클라우드 설정도 필요 없습니다. 그냥 GUI를 열고 사용하면 됩니다.

### 5-1. 어디에 저장되나요?

```
<■■■■ ■■■>/data/osmu_workbook.xlsx
```

이 파일 한 개에 두 시트(keyword\_pool, content\_db)가 들어 있습니다. Excel/Numbers 에서 더블클릭으로 열어 직접 보거나 편집할 수 있어요.

### 5-2. 직접 엑셀에서 수정해도 되나요?

[사용자 작업] GUI를 닫고 → Excel에서 osmu\_workbook.xlsx 열기 → 편집 → 저장 → GUI 다시 실행

[사용자 작업] GUI 사용 중에는 Excel을 닫아두는 게 안전합니다 (동시 쓰기 충돌 방지)

[사용자 작업] 헤더 행(1행)은 변경하지 마세요 — 컬럼 순서가 코드와 맞물려 있습니다

### 5-3. 백업 · 이동

이 파일 하나만 복사하면 모든 데이터가 백업됩니다. 다른 컴퓨터로 옮길 때도 같은 위치에 두면 즉시 인식.

## 6. 구글 시트와 양방향 동기화

이 기능을 쓰려면 한 번만 구글 서비스 계정 자격증명을 발급받아야 합니다. 10~15분이면 끝나며, 한 번 설정하면 영구적으로 동작합니다.

### 6-1. 구글 클라우드 콘솔에서 서비스 계정 만들기

[사용자 작업] console.cloud.google.com 접속 → 새 프로젝트 만들기 (이름 자유)

[사용자 작업] 왼쪽 메뉴 → ‘API 및 서비스’ → ‘라이브러리’ → ‘Google Sheets API’ 검색 → 사용 설정

[사용자 작업] 같은 단계에서 ‘Google Drive API’ 도 사용 설정 (시트 자동 생성에 필요)

[사용자 작업] 왼쪽 메뉴 → ‘IAM 및 관리자’ → ‘서비스 계정’ → ‘만들기’

[사용자 작업] 이름 적당히 (예: osmu-bot) → 만들기 → 역할 ‘기본’ → ‘편집자’ → 완료

[사용자 작업] 방금 만든 서비스 계정 클릭 → ‘키’ 탭 → ‘키 추가’ → ‘새 키 만들기’ → JSON → JSON 파일 자동 다운로드

[사용자 작업] 다운받은 JSON 파일을 프로젝트 폴더 하위 credentials/service\_account.json 으로 이동

### 6-2. 빈 구글 시트 만들고 권한 부여

[사용자 작업] sheets.new 접속해 빈 시트 1개 만들기

[사용자 작업] 오른쪽 위 ‘공유’ → 받는 사람에 서비스 계정 이메일 입력 (JSON 파일의 client\_email 값) → 권한 ‘편집자’ → 보내기

[사용자 작업] 시트 URL 의 /d/ 다음 문자열을 복사 (이게 OSMU\_SHEET\_ID)

### 6-3. GUI에 연결

‘설정’ 화면에서 다음을 입력합니다:

[사용자 작업] ‘저장 위치’ → ‘양방향 동기화 (권장)’ 선택

[사용자 작업] ‘구글 시트 ID’ → 6-2에서 복사한 문자열 붙여넣기

[사용자 작업] ‘구글 서비스 계정 키 파일 경로’ → credentials/service\_account.json

[사용자 작업] ‘저장하기’ 클릭

### 6-4. 동작 확인

[사용자 작업] ‘키워드 풀 관리’ 화면 상단에 켜짐 으로 바뀜

[사용자 작업] ‘시트로 업로드’ 클릭 → 구글 시트 새로고침 시 데이터 보임

[사용자 작업] 구글 시트에서 직접 한 줄 수정 → ‘시트에서 새로고침’ 클릭 → 변경 반영 확인

### 충돌 정책

두 곳에서 동시에 같은 키워드를 수정하면 마지막에 저장된 쪽이 이깁니다(last-write-wins). 현 버전은 단일 운영자 기준으로 설계됐으며, 멀티 사용자 동시 편집은 향후 버전에서 다룰 예정입니다.

## 7. GitHub에 올리기

### 7-1. Git 초기 설정 (한 번만)

```
git config --global user.name "■■ ■■"  
git config --global user.email "■■ ■■■"
```

### 7-2. 저장소 초기화 + 첫 커밋

```
cd osmu_keyword_researcher  
git init && git branch -M main  
git add . && git commit -m "feat: ■■ ■■"
```

### 7-3. GitHub 저장소 만들고 푸시

[사용자 작업] github.com/new 에서 빈 저장소 만들기

[사용자 작업] README/.gitignore/license 옵션 모두 체크 해제 (이미 로컬에 있음)

[사용자 작업] 표시되는 명령 그대로 실행:

```
git remote add origin https://github.com/<■■■■>/osmu-keyword-researcher.git  
git push -u origin main
```

### 7-4. 자격증명 안 올라갔는지 확인

```
git ls-files | grep -E "credentials|\.env|service_account" || echo "■ ■■"
```

## 8. GitHub Codespaces로 실행하기

[사용자 작업] GitHub 저장소 페이지 → 초록색 ‘<> Code’ 버튼 → ‘Codespaces’ → ‘Create codespace on main’

[사용자 작업] 1~2분 후 브라우저에서 VS Code 자동 오픈

[사용자 작업] 터미널에 `python main.py` 입력 → 8501 포트 자동 미리보기

## 9. GitHub Actions 자동 테스트

`.github/workflows/tests.yml` 이 이미 들어 있어, main 브랜치 푸시 / PR / 수동 트리거 시 자동 실행됩니다.

[Claude 완료] Python 3.10 / 3.11 / 3.12 + macOS 매트릭스 단위 테스트

[Claude 완료] 시연 시나리오 ① · ② 스모크 테스트

[Claude 완료] `py_compile` 로 전체 파일 구문 검사

[사용자 작업] 결과 확인은 GitHub 저장소 → Actions 탭

## 10. Codespace Prebuild & Secret 관리

Prebuild 는 의존성 설치를 미리 해두는 캐시. Codespace 시작 시간을 1~2분에서 10~20초로 단축합니다.

[Claude 완료] devcontainer.json 을 prebuild 친화적으로 분리 (onCreate / postStart)

[Claude 완료] on-create.sh, post-start.sh 작성 — 의존성 · 자격증명 자동 처리

[사용자 작업] GitHub 저장소 → Settings → Codespaces → Prebuild configurations → ‘Set up prebuild’

[사용자 작업] 분기 main, 트리거 ‘Every push’, Region 선택 → Save

### 10-1. 구글 시트 자격증명을 Secret 으로 등록

[사용자 작업] GitHub 저장소 → Settings → Secrets and variables → Codespaces → New repository secret

[사용자 작업] 이름 OSMU\_SHEET\_ID, 값 = 시트 ID 문자열

[사용자 작업] 이름 GOOGLE\_APPLICATION\_CREDENTIALS\_JSON, 값 = JSON 파일 전체 내용

[사용자 작업] 다음 Codespace 시작부터는 자동으로 자격증명 풀어져 있음

## 11. 자주 만나는 문제 (FAQ)

Q. ‘command not found: pip’ 가 떠요

A. Mac은 pip3 만 있습니다. 가상환경 활성화 후엔 pip 도 됩니다.

Q. ‘externally-managed-environment’ 에러

A. 최신 Homebrew Python 보안 정책. 가상환경(venv)을 만들어 그 안에서 설치하세요.

Q. ‘Permission denied’ — run\_mac.command 실행 안 됨

A. 터미널에서 `chmod +x ui/run_mac.command` 한 번 실행. 또는 `python main.py` 로 우회.

Q. Excel에서 파일 열어두면 GUI 가 멈춰요

A. Excel이 파일 잠금을 걸기 때문입니다. GUI 사용 중에는 Excel을 닫아주세요. 또는 GUI에서 ‘엑셀로 내려받기’ 로 사본을 받아 보세요.

Q. 구글 시트 연결이 자꾸 실패해요

A. ① 서비스 계정 이메일을 시트 ‘공유 → 편집자’ 로 추가했는지 ② Google Sheets API + Google Drive API 둘 다 사용 설정했는지 ③ JSON 파일 경로가 맞는지 확인.

Q. ‘최근에 비슷한 주제...’ 메시지가 떠요

A. 정상 동작입니다. ‘설정 → 동일 주제 재사용 간격’ 을 줄이면 풀립니다.

Q. Codespace 가 너무 느려요

A. 10장의 Prebuild 활성화 단계를 따라 한 번 설정해두면 다음부터 10~20초로 줄어듭니다.

Q. 자격증명이 GitHub에 올라갈까봐 걱정돼요

A. .gitignore가 .env / credentials/ / \*.json 패턴을 모두 막고 있습니다. 7-4 절 점검 명령으로 확인하세요.

## 12. 확장 방향 — 다음 단계

현재 모듈은 키워드 단계를 책임집니다. 선택된 키워드는 content\_db 에 ‘대기중’ 으로 등록되며, 다음 단계 모듈이 이를 받아서 자동 발행까지 이어집니다.

| 단계        | 역할                        | 출력           |
|-----------|---------------------------|--------------|
| 1. 글감 수집  | OpenClaw로 기사 3개 크롤링       | raw_content  |
| 2. 이미지 수집 | Unsplash API 로 이미지 URL 3개 | image_urls   |
| 3. 글 생성   | Claude가 SEO 최적 본문 작성      | refined_post |
| 4. PM 검토  | Slack 미리보기 → 승인/수정/거절     | status 변경    |
| 5. 자동 발행  | 티스토리 API → 게시             | platform_url |

---

끝까지 읽어주셔서 감사합니다. 사용 중 문제는 GitHub Issues 로 알려주세요.